

FOUNDATIONS · FROM A NEURON TO A NETWORK

Deep Learning Foundations

How networks learn: neurons and activations, the forward pass, gradient descent and back-propagation, loss functions, architecture design, and the tools that make it real.

Neurons

XOR

Gradient descent

Back-prop

PyTorch



00 WHAT WE'LL COVER

The road map



Recall NN & deep learning

01

why it works, where it's used



Neurons & forward pass

02

the building block, stacked



Learning XOR

03

why hidden layers matter



Activations & loss

04

ReLU, softmax, cross-entropy



Gradient descent & back-prop

05

how weights actually update



Design & tooling

06

hidden units, PyTorch, Colab

✦ IN PLAIN ENGLISH

Neural-network basics

Neural network

Layers of tiny units that learn patterns from examples.

e.g. *spotting a cat in a photo*

Neuron

One unit: weigh the inputs, add a bias, squash the result.

e.g. *a mini decision-maker*

Bias

A nudge that shifts how easily the neuron fires.

e.g. *its baseline eagerness*

Deep learning

Neural nets with many layers that learn features themselves.

e.g. *no hand-written rules, just data*

Weight

How much a given input matters to a neuron.

e.g. *turning a volume knob up or down*

Activation function

The squash that lets networks learn curves, not just lines.

e.g. *ReLU, sigmoid, tanh*

✦ IN PLAIN ENGLISH

Activations & outputs

Sigmoid

Squashes any number into 0–1, like a probability.

e.g. *“how likely is this spam?”*

ReLU

Keep positives, zero out negatives — simple and fast.

e.g. *the default in modern nets*

Layer

A row of neurons — input, hidden, and output layers.

e.g. *one stage of an assembly line*

Tanh

Like sigmoid but ranges -1 to 1 , and is zero-centred.

e.g. *balanced positives & negatives*

Softmax

Turns scores into percentages that add to 100%.

e.g. *70% cat, 20% dog, 10% fox*

Feedforward

Data flows one way, input to output, no loops.

e.g. *a straight pipeline*

✦ IN PLAIN ENGLISH

How learning works

Loss / error function

A score for how wrong the model's prediction is.

e.g. lower is better

Cross-entropy

Classification loss that punishes confident mistakes hard.

e.g. used for categories / yes-no

Gradient descent

Repeatedly step downhill to shrink the loss.

e.g. rolling a ball into a valley

MSE

Average of the squared gaps between prediction and truth.

e.g. used when predicting numbers

Gradient

The slope showing which way increases the error.

e.g. which direction is uphill

Learning rate

How big each downhill step is.

e.g. too big overshoots, too small crawls

✦ IN PLAIN ENGLISH

Back-prop & training

Back-propagation

Sends the error backwards to update every weight.

e.g. assigning blame layer by layer

Forward pass

Running inputs through the net to get a prediction.

e.g. one trip down the pipeline

Parameter

A weight or bias the model learns.

e.g. big nets have millions

Chain rule

Multiply local slopes to get the overall slope.

e.g. gears passing motion along

Epoch

One full pass through all the training data.

e.g. reading the whole textbook once

Vanishing gradient

Gradients shrink to almost nothing in deep nets.

e.g. why ReLU replaced sigmoid

✦ IN PLAIN ENGLISH

Design & tools

Hidden units

Neurons in the middle layers — more means more power.

e.g. *the network's working 'brain cells'*

Underfitting

Model too simple to capture the pattern.

e.g. *a straight line through a curve*

GPU

A chip that does the huge parallel math fast.

e.g. *Colab gives you one free*

Capacity

How complex a pattern the model can represent.

e.g. *a bigger net fits wigglier curves*

Overfitting

Model memorizes noise instead of the real pattern.

e.g. *great in practice, poor on the exam*

Framework

Software that auto-computes back-prop for you.

e.g. *PyTorch, TensorFlow, Keras, JAX*

What deep learning is — and why it works

The idea: instead of hand-coding rules, show a layered network millions of examples and let it learn the patterns itself. Stack enough layers and it discovers simple features, then combines them into complex ones.



Vision

faces, medical scans,
self-driving



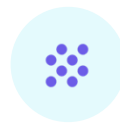
Language

translation, chatbots,
search



Speech & audio

voice assistants,
transcription



Science

drug discovery,
weather, genomics

A running example: predicting a house's price

size (sq ft)	bedrooms	location /10	age (yrs)	price
1,500	3	8	10	\$420k
2,300	4	6	2	\$510k
950	2	9	40	\$380k
3,100	5	7	5	\$690k

inputs = features (what we know) → output = price (what we predict)

Why a neural net?

- Price depends on non-linear mixes: a big house in a weak area, or an old house *newly renovated*
- The network learns these interactions from thousands of past sales — no hand-written rules
- We'll follow THIS example through every concept ahead.

What is Neural Network ?

A neural network is a type of machine learning algorithm inspired by the human brain.

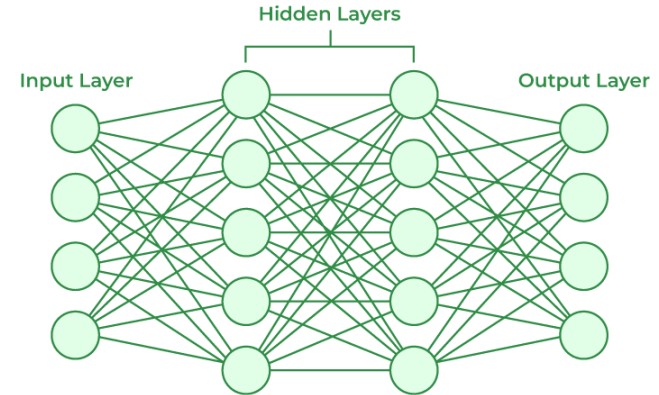
It's a powerful tool that excels at solving complex problems more difficult for traditional computer algorithms to handle, such as image recognition and natural language processing.

Composed of interconnected nodes called neurons, neural networks arrange these units in layers. Each neuron receives input from others, processes it, and transmits an output to other neurons.

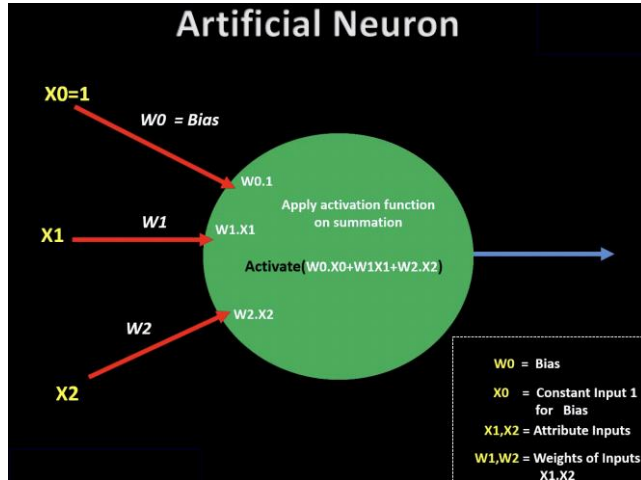
Connections between neurons have associated weights, signifying the connection strength.

During training, the network adjusts these weights to refine its performance on a given task.

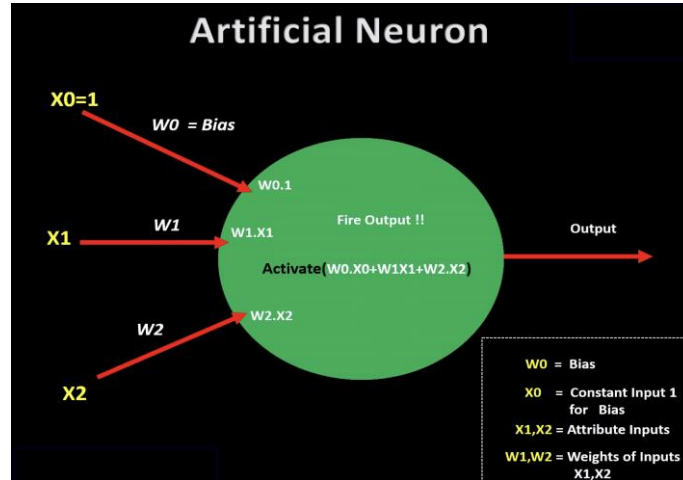
This learning process allows them to make predictions and recognize patterns, driving their wide adoption in diverse applications like image recognition, natural language processing, and machine translation.



What is Neural Network ?



Linear Calculation Summarization



Activation Function

- It takes certain inputs and weights
- Applies dot product on respective inputs & weights and apply summation
- Apply some transformation using activation function on the above summation
- Fires output

What is a Neural Network?

For Example:

You love to bike to work. You want a Neural Network which makes decision for you. Your decision depends on two factors : Weather, and it must be a Weekday

We provide the information to Neural Network, for you Weekday is more important than Weather. That is where Weights come into picture. A higher weight to Weekday parameter, will make the Neural Network consider it more important compared to the weather factor

Let us purposefully set suitable weights for 2 for the Weather and 6 for weekday and say if it is nice and sunny we give it as 1, and for weekday it is 1

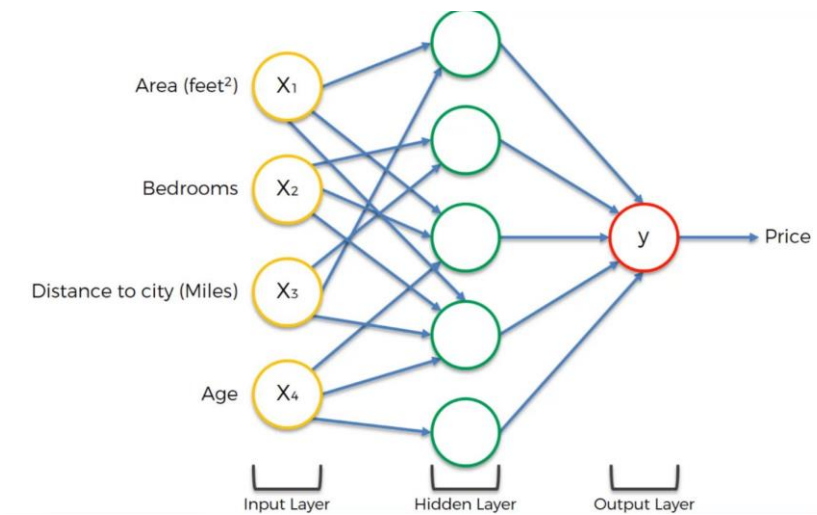
So Now, total = (nice..weather ?) $\times$ (weight1 for weather) + (weekday) $\times$ (weight2)

$$= (1 \times 2) + (1 \times 6) = 8$$

This calculation is the Linear Combination. What does this 8 mean? The Neural Network will give the output 0 which is Stay at Home or 1 Go to work.

Let us say the Threshold value is 5, which means if the Linear Combination value come greater than 5, we get output 1 from the Neural Network meaning We go to work, if it is less than 5 output from Neural Network is zero, you stay at home. This conversion of Linear Combination to Output as 1 or 0 is done by Activation Function.

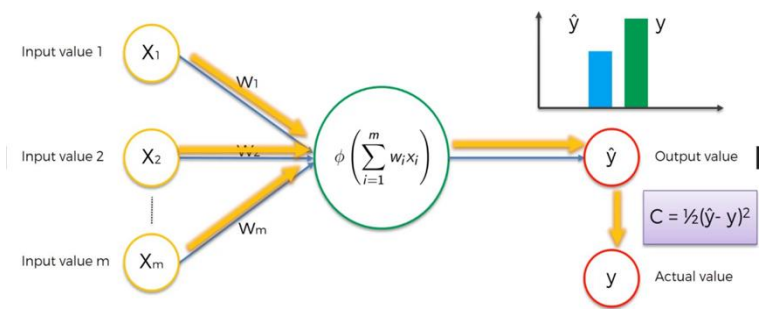
What is a Neural Network ?



Let us Say we want to build a Neural Network to predict price of property

Features we use are Area, Bedrooms, Distance to city , Age

Area (sq. feet)	Bedrooms	Distance to city (miles)	Age	Actual Price
600	2	20	10	50,00,000



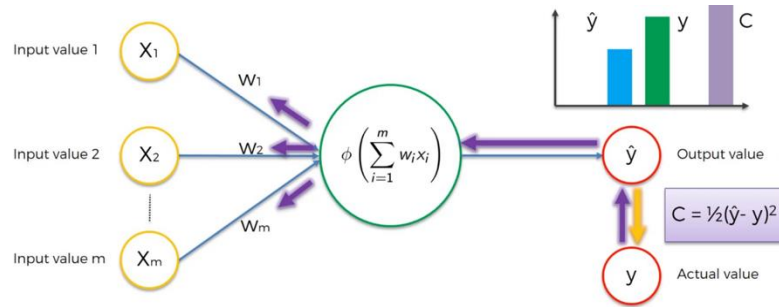
When training the Neural Network, based on observed data, it starts to give it's predicted output value $\hat{y}$ for our training dataset

We already have the actual value of the price y in training dataset

So, a function called Cost Function helps us in minimizing the error between actual and predicted values



What is Neural Network ?



Based on the Cost Function Calculation, the error is propagated back to adjust the weights, for the corrections to happen.

This is Backpropagation.

The process is reiterated again and again till the Cost Function is minimized to give minimum error between predicted and actual value in the training dataset

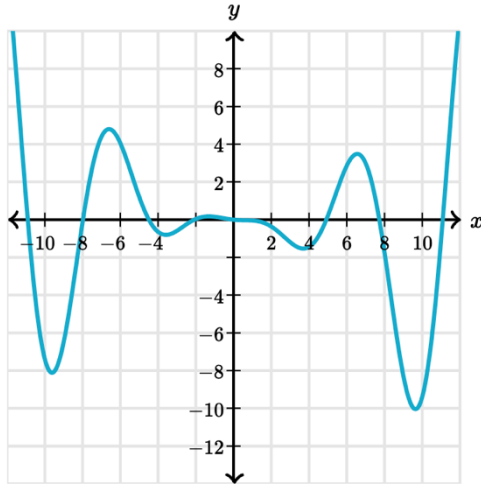
Area (sq. feet)	Bedrooms	Distance to city (miles)	Age	Actual Price	Predicted Price
600	2	20	10	50,00,000	39,00,000

Gradient descent is a mathematical optimization algorithm that helps us minimize Cost Function, which helps to come to a step of the optimized value of weights

What is Network ?

Let us Consider a function for which we want to find a minima
:

$$\text{Consider the function } f(x) = \frac{x^2 \cos(x) - x}{10}.$$

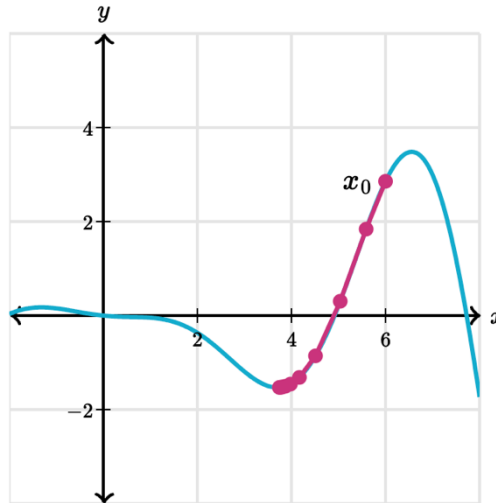


As we can see from the graph, this function has many local minima. Gradient descent will try to find the minima based on initial guess and a step size

What is a Neural Network ?

If we choose $X_0 = 6$ and $\alpha = 0.2$ (step size), for example, gradient descent moves as shown in the graph below. The first point is X_0 , and lines connect each point to the next in the sequence. After only 10 steps, we have converged to the minimum near $X = 4$.

10 Steps being (Step 1. $6 - 0.2 = 5.8$, Step 2. $5.8 - 0.2 = 5.6$ Step 9. $4.4 - 0.2 = 4.2$, Step 10. $4.2 - 0.2 = 4$)



What is Neural Network?

Examples of neural networks

Beyond broad applications, here are specific examples of neural networks in action:

- Handwriting Recognition:** Digitizing handwritten documents or automating mail sorting benefits from neural networks' ability to recognize handwritten characters.
- Facial Recognition:** Security systems, social media tagging, and personalized advertising utilize the power of neural networks for facial recognition.
- Medical Image Analysis:** Detecting anomalies in medical images like X-rays and MRI scans is made possible through neural networks, aiding disease diagnosis and treatment planning.

What is Neural Network ?

Types of neural networks

We categorize neural networks based on their architecture, training methods, and uses. Common types include:

- **Feedforward Neural Networks:** This basic type processes data linearly from input to output, without loops. They're commonly used for straightforward tasks like classification and regression.
- **Recurrent Neural Networks (RNNs):** Designed for sequential data like time series or natural language, RNNs use a feedback loop to retain memories of previous inputs, making them suitable for data with a time component.
- **Convolutional Neural Networks (CNNs):** Specifically designed for image-related tasks, CNNs extract features from images using a convolutional layer, which identifies patterns like edges and shapes.
- **Generative Adversarial Networks (GANs):** Composed of a generator and a discriminator, GANs pit these two components against each other. The generator creates data while the discriminator assesses its authenticity. This adversarial process results in the generator producing increasingly realistic data, often used for generating images, videos, and audio.

What is Neural Network ?

Importance of neural networks

Neural networks are becoming an essential tool for many businesses and organizations.

Here are some reasons why they are so important:

- Automation:** Automating tasks that were previously done by humans, such as customer service, data analysis, and image processing can be assisted by neural networks. This can save businesses time and money.
- Improved Decision-Making:** Businesses can make better decisions by providing insights that would be difficult or impossible to obtain using traditional methods.
- Increased Efficiency:** Business processes can be improved by automating tasks, reducing errors, and improving decision-making.
- New Products and Services:** Businesses are enabled to create new products and services that would not be possible without AI.

What is Neural Network ?

Neural network applications and uses

Neural networks can be used in a wide range of applications, including:

Image recognition

Identifying objects in images finds uses in self-driving cars, social media platforms, and medical diagnoses.

Natural Language Processing (NLP)

Machine translation, text summarization, and sentiment analysis are made possible with neural networks, powering tools like Google Translate and chatbots.

Financial modeling

Neural networks may help financial institutions by analyzing historical financial data, and identifying trends and patterns that can be used to help consider investment decisions. They may also be used for risk management, fraud detection, and portfolio optimization.

Medical diagnosis

With the assistance of neural networks, diagnosing diseases and predicting patient outcomes may be possible through using medical data to identify patterns that are associated with specific diseases. This may help doctors make more accurate diagnoses and develop personalized treatment plans.

What is Neural Network ?

Robotics

The movements of robots may be controlled and trained to perform complex tasks, such as assembly, welding, and surgery. Neural networks can also be used in robot navigation and path planning.

Self-driving vehicles

Powering the vision systems in self-driving cars can be done by identifying objects like pedestrians, traffic lights, and other vehicles, and then making decisions about how to drive safely. Neural networks can also help with tasks like lane keeping, adaptive cruise control, and collision avoidance.

Fraud detection

Detecting fraudulent transactions can be done by analyzing data from credit card transactions, where these networks identify patterns that are associated with fraud. This can help banks and other financial institutions to prevent fraudulent activity.

Spam Filtering

Filtering spam emails can be done by analyzing the content of emails, where neural networks can help identify patterns that are associated with spam. This may help users to avoid receiving unwanted emails.

What is Neural Network ?

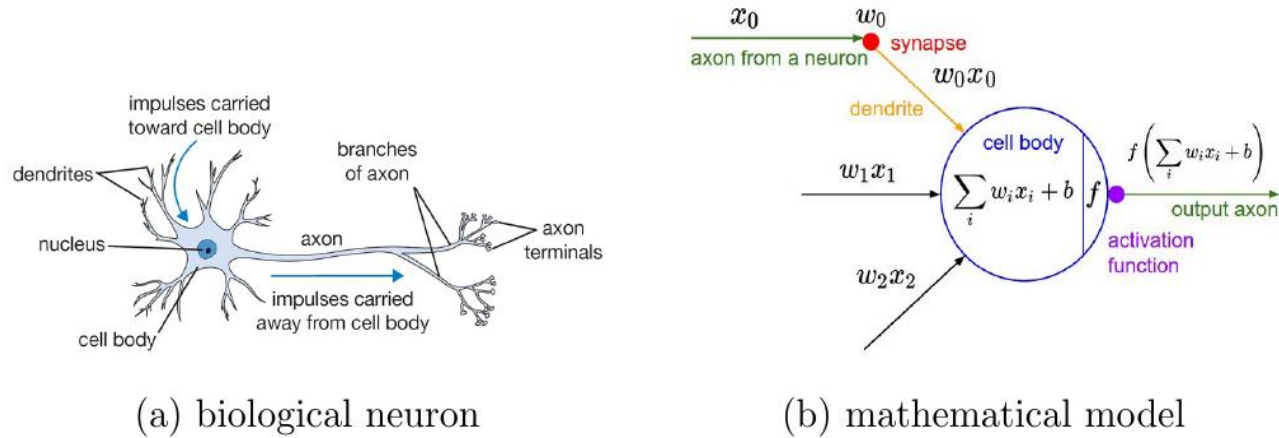
Neural network advantages

There are several advantages neural networks have over traditional computer algorithms, including:

- Adaptability:** Neural networks can learn and adapt to new data, making them more flexible and robust than traditional algorithms.
- Non-Linearity:** Complex non-linear relationships in data can be learned, which is something that traditional algorithms struggle with.
- Parallel Processing:** Neural networks can be parallelized, meaning that they can be run on multiple processors simultaneously, making them faster and more efficient.
- Fault Tolerance:** They are tolerant of noise and missing data, making them more reliable than traditional algorithms.
- Generalization:** Can generalize to new data that they have not been trained on, making them more versatile than traditional algorithms.

Human Brain

The main computational units of the brain are known as neurons; in the human nervous system, approximately 86 billion neurons can be found, all of which are connected by synapses. The following diagram shows a biological neuron and the mathematical model that draws inspiration from it:



Representation of the biological neuron, (a), on the left and its mathematical model, (b), on the right.

Human Brain

Biological neurons are made up of the following:

Dendrites: Minor fibers that carry information, in the form of an electric signal, from the outside to the nucleus.

Synapses: These are the connection points among neurons. Neurons receive input signals on the synapses that are connected to the dendrites.

Nucleus: This receives the signals from the dendrites, elaborates on them, and produces a response (output signal) that it sends to the axon.

Axon: The output channel of the neuron. It can be connected to other neuron synapses.

Human Brain

- Each neuron receives input signals from its dendrites and transports them to the nucleus where they are processed;
- Dendrites process the signals, thereby integrating (adding up or combining) excitation and inhibition from every input synapse.
- The nucleus receives the integrated signals and adds them. If the final sum is above a certain threshold, the neuron fires and the resulting information is carried down through the axon and thus to any other connected neuron.
- The amount of signal that's transmitted among neurons depends on the strength of the connections. It is the arrangement of the neurons and the strength of these synapses that establish the function of the neural network.
- The learning phase of biological neurons is based on the modification of the output signal generated by the nucleus over time, as a function of certain types of input signals. Neurons specialize themselves in recognizing certain stimuli during their lifetime.

DEEP LEARNING · THE NON-LINEAR SPARK

Activation Functions

What they are, how each one behaves, and — with real use cases — which one to reach for and when.

Sigmoid

Tanh

ReLU

Leaky / ELU

GELU

Softmax



Why a network needs a non-linearity

The problem: stack linear layers and the whole network collapses back into ONE straight line — no matter how deep. The activation is the bend that lets networks model curves, corners, and complex patterns.

Where it sits in a neuron

$$z = w \cdot x + b \quad \rightarrow \quad a = f(z)$$

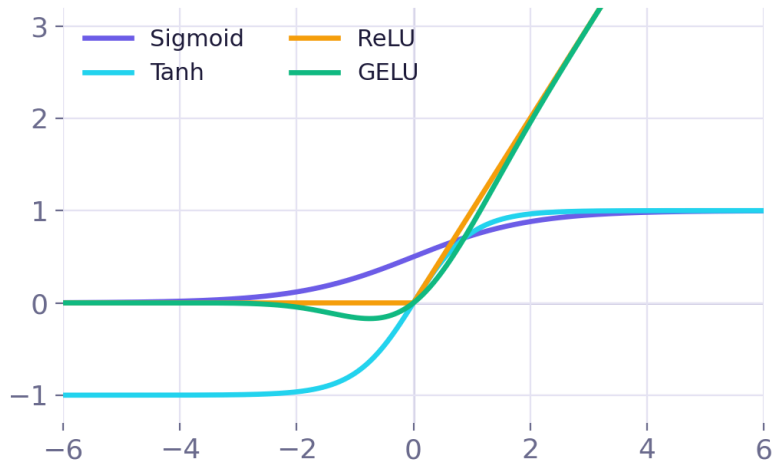
- First the weighted sum z (a linear step).
- Then $f(z)$ — the activation — adds the non-linearity.
- Stack these and the network can bend anywhere.

Two jobs of an activation

- Add non-linearity — the whole point.
- Keep values in a useful range (e.g. 0–1 as a probability).
- Pass a healthy gradient so learning can flow backward.

02 THE LINEUP

The activations at a glance



four common activations, overlaid

Sigmoid / Tanh

smooth S-curves — squash to a fixed range

ReLU & variants

the fast, deep-net workhorse

GELU / Swish

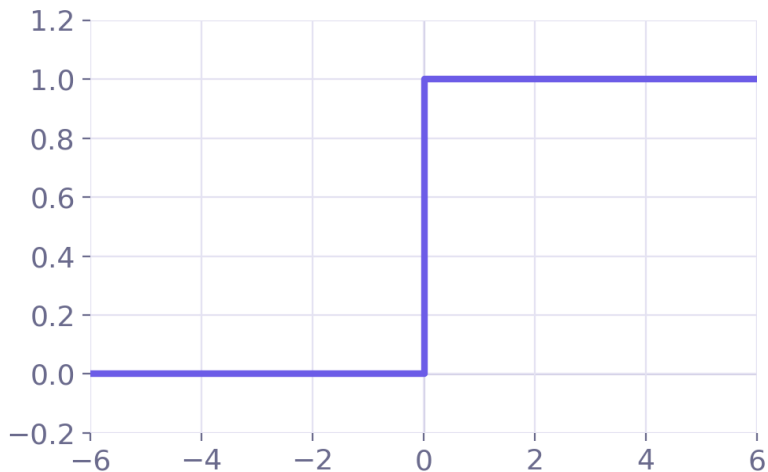
smooth, modern — powers Transformers

Softmax

the output layer for multi-class

03 ACTIVATION FUNCTION

Step / Threshold



the original perceptron output — 0 or 1

$$f(x) = 1 \text{ if } x \geq 0, \text{ else } 0$$

Range: $\{0, 1\}$

USE IT FOR

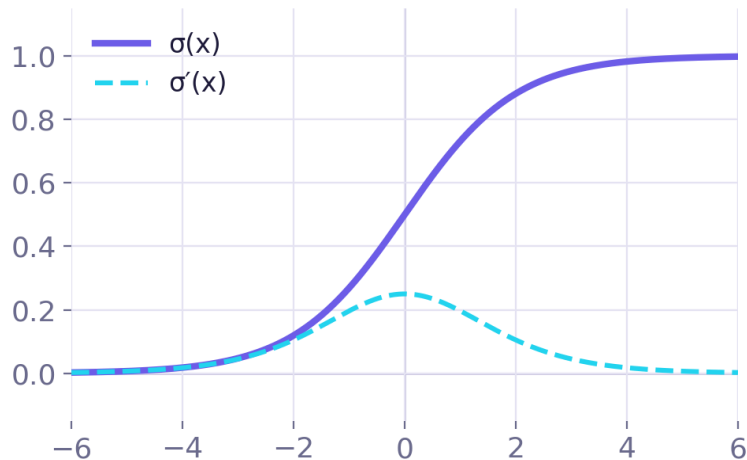
- ✓ Teaching the basic perceptron idea
- ✓ Simple hard yes/no decisions

WATCH OUT

- ! Gradient is 0 everywhere → can't train with gradient descent (why it's obsolete)

04 ACTIVATION FUNCTION

Sigmoid (logistic)



solid $\sigma(x)$ · dashed $\sigma'(x)$ — note the flat tails

$$\sigma(x) = 1 / (1 + e^{-x})$$

Range: (0, 1)

USE IT FOR

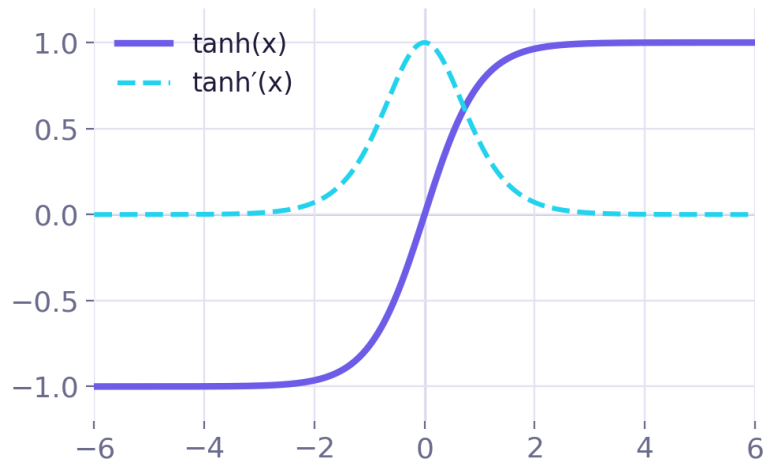
- ✓ OUTPUT of a binary classifier (a probability)
- ✓ Gates in LSTM / GRU cells
- ✓ e.g. spam probability, churn, click-through

WATCH OUT

- ! Saturates → vanishing gradient in deep nets; not zero-centred

05 ACTIVATION FUNCTION

Tanh



solid $\tanh(x)$ · dashed $\tanh'(x)$ — zero-centred

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Range: (-1, 1)

USE IT FOR

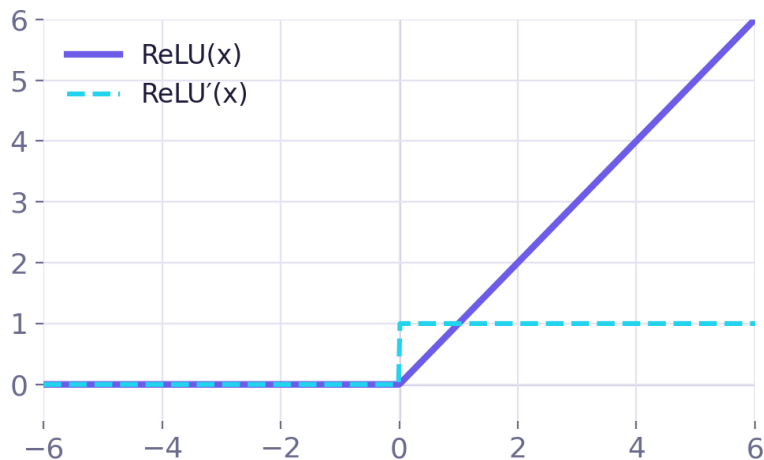
- ✓ Hidden layers of classic RNNs
- ✓ When you want zero-centred outputs
- ✓ LSTM candidate / cell values

WATCH OUT

- ! Still saturates at the extremes like sigmoid

06 ACTIVATION FUNCTION

ReLU



solid $\text{ReLU}(x)$ · dashed gradient (0 or 1)

$$f(x) = \max(0, x)$$

Range: $[0, \infty)$

USE IT FOR

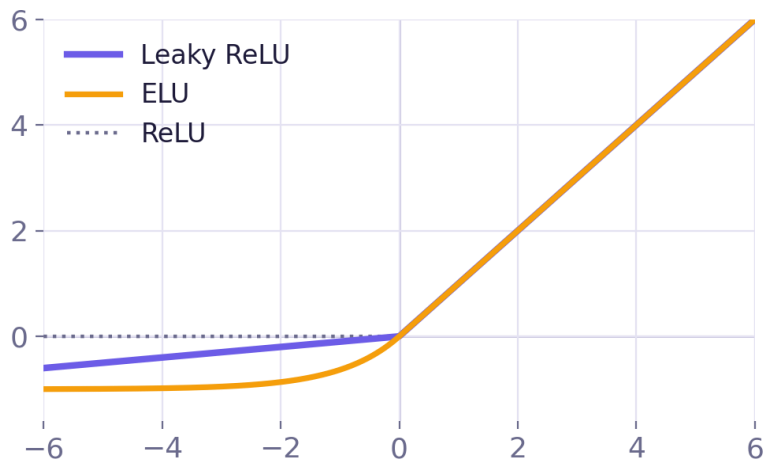
- ✓ The DEFAULT for hidden layers of deep nets & CNNs
- ✓ Fast, non-saturating for $x > 0$
- ✓ e.g. image-classification hidden layers

WATCH OUT

- ! "Dead ReLU": a unit stuck at 0 stops learning; not zero-centred

07 ACTIVATION FUNCTION

Leaky ReLU / ELU



a small slope for $x < 0$ keeps units alive

Leaky: $\max(0.01 \cdot x, x)$ ELU:
 $\alpha(e^x - 1)$

Range: $(-\infty, \infty)$

USE IT FOR

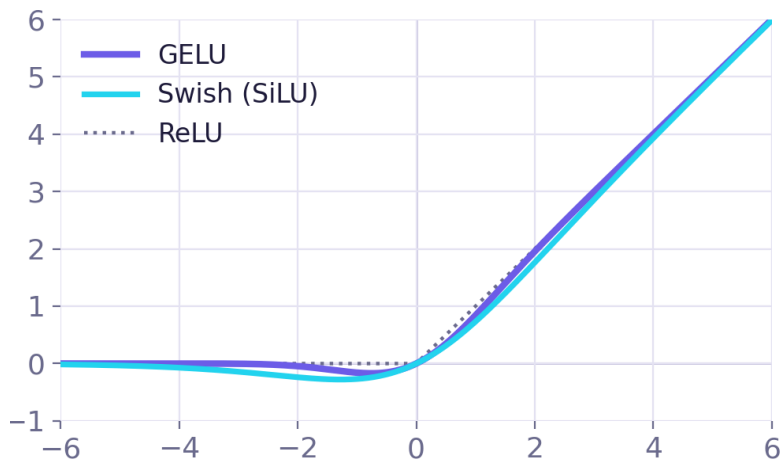
- ✓ Deep nets where too many ReLUs are dying
- ✓ ELU can speed up convergence
- ✓ A safe drop-in replacement for ReLU

WATCH OUT

- ! Adds a small hyper-parameter (the negative slope α)

08 ACTIVATION FUNCTION

GELU / Swish (SiLU)



smooth, modern — a softened ReLU

GELU: $x \cdot \Phi(x)$

Swish: $x \cdot \sigma(x)$

Range: $\approx (-0.17, \infty)$

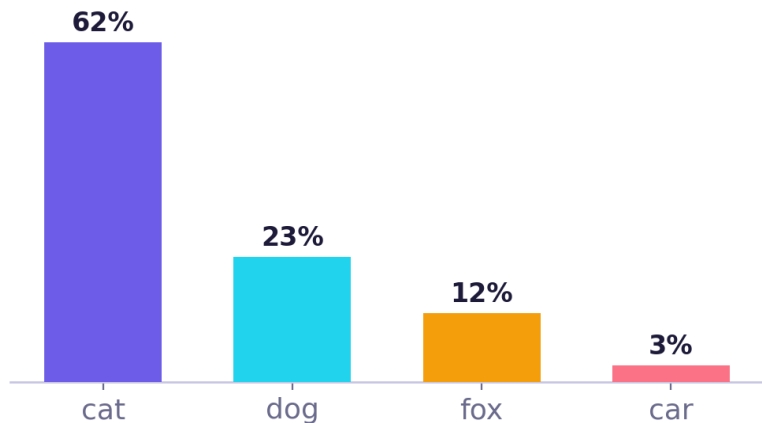
USE IT FOR

- ✓ Hidden layers of modern Transformers
- ✓ GELU powers BERT & GPT; Swish in vision nets
- ✓ Often edges out ReLU on accuracy

WATCH OUT

- ! Slightly more compute than plain ReLU

Softmax



turns raw scores into probabilities that sum to 100%

$$\text{softmax}(x)_i = e^{x_i} / \sum e^{x_j}$$

Range: (0, 1), sums to 1

USE IT FOR

- ✓ OUTPUT layer of a MULTI-class classifier
- ✓ Next-token probabilities in language models
- ✓ e.g. image classes, document categories

WATCH OUT

- ! Output layer only — never a hidden layer; use the stable (max-subtracted) form

The activation cheat-sheet

Function	Range	Best for	Watch out
Sigmoid	$(0, 1)$	binary output · gates	vanishing gradient
Tanh	$(-1, 1)$	RNN hidden layers	still saturates
ReLU	$[0, \infty)$	default hidden layers	dead units
Leaky ReLU / ELU	$(-\infty, \infty)$	when ReLUs die	extra slope α
GELU / Swish	$\approx(-0.17, \infty)$	Transformer hidden layers	a bit more compute
Softmax	$(0,1) \Sigma=1$	multi-class OUTPUT	outputs only

11 DECISION GUIDE

Which activation should I use?

HIDDEN layers	ReLU (then try GELU / Leaky ReLU)	<i>start simple and fast; upgrade if needed</i>
BINARY output	Sigmoid	<i>one probability between 0 and 1</i>
MULTI-CLASS output	Softmax	<i>a probability per class, summing to 1</i>
REGRESSION output	None (linear)	<i>predict a raw number — e.g. a price</i>

Rule of thumb: ReLU everywhere hidden, and let the TASK pick the output activation.

12 USE CASES IN THE WILD



Real systems, real choices

Spam filter

hidden: ReLU · output: Sigmoid

one probability: spam or not

Image classifier

hidden: ReLU · output: Softmax

cat / dog / car ... one winner

House-price model

hidden: ReLU · output: linear

a raw dollar amount, unsquashed

GPT / BERT (LLMs)

hidden: GELU · output: Softmax

smooth hidden units, next-token probs

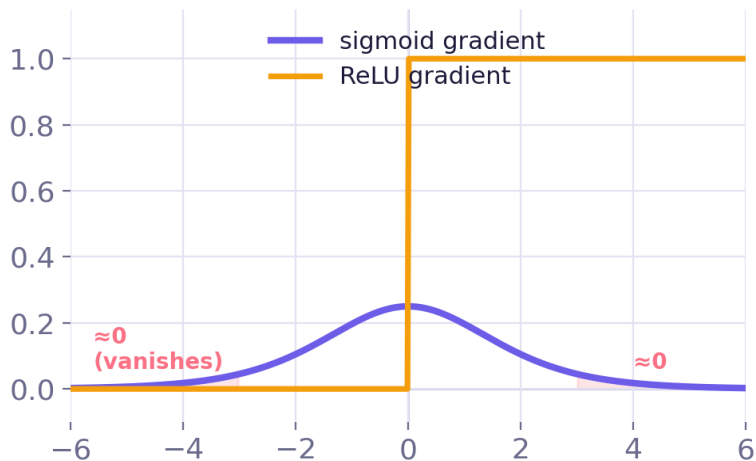
LSTM (time series)

gates: Sigmoid · cell: Tanh

sigmoid decides, tanh proposes values

13 PITFALLS

Two traps to know



sigmoid's gradient vanishes at the tails; ReLU's stays 1

Vanishing gradient

Sigmoid/tanh flatten far from 0, so their gradient $\rightarrow 0$. In deep nets the backward signal shrinks to nothing and early layers stop learning. Fix: use ReLU-family activations.

Dead ReLU

A ReLU that only ever sees negative inputs outputs 0 forever and its gradient is 0 — it never recovers. Fix: Leaky ReLU / ELU, or a smaller learning rate.



KEY TAKEAWAYS

Five things to remember

- 1 Activations add the non-linearity — without them, depth is pointless.
- 2 Hidden layers: default to ReLU; upgrade to GELU / Leaky ReLU if needed.
- 3 Output layer: the task decides — Sigmoid, Softmax, or linear.
- 4 Sigmoid/tanh saturate → vanishing gradients in deep nets.
- 5 Match the activation to where it lives and what the task needs.



The tools that make it practical

PyTorch

by Meta

flexible, dynamic — most research

TensorFlow

by Google

production & deployment, mobile

Keras

by Google

high-level, beginner-friendly

JAX

by Google

NumPy + autograd + speed, TPUs

All give you autograd (free back-prop) + GPU speed. Run on a GPU/TPU — Google Colab offers one free in the browser, so every lab in this course trains in minutes with zero setup.



H A N D S - O N

Now go build a brain



Deep Learning Playground

7 tabs — neuron, activations, XOR, gradient descent, back-prop



DL Labs 1–5

neurons · XOR · gradient descent · back-prop · a real net in PyTorch



Train on free GPU

every notebook runs in Google Colab in minutes



How a Neural Network Learns XOR

XOR is a tiny puzzle a plain model can't solve — until you add a "hidden layer." Here's why, in plain terms.



THE RULE

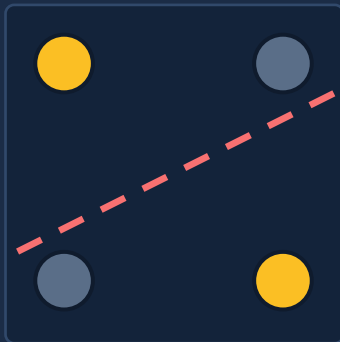
Two switches in. One light out.

A	B		Light
0	0	→	
0	1	→	
1	0	→	
1	1	→	

The light turns **ON only when the two switches disagree.**



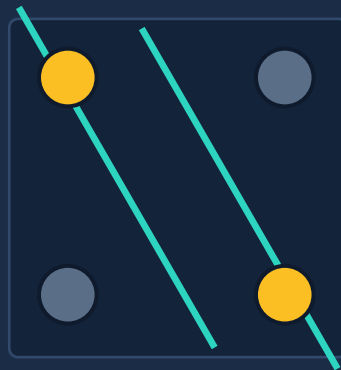
THE PROBLEM



No single straight line can put both amber dots on one side. A simple model gets stuck.



THE SOLUTION



Two simple boundaries — learned by the hidden layer — combine to fence in exactly the amber dots.